Programare Web / Dezvoltarea Aplicațiilor Web
Tehnologia Informației, anul IV · Informatică, anul II · Anul universitar 2026–2027, semestrul I

LABORATORUL 06

Backend cu Symfony: rute, controllere și răspunsuri JSON

Trecem pe server: scriem primul nostru controller, cu trei endpoint-uri care întorc JSON, le testăm în browser, Swagger și Postman și învățăm să citim codurile de stare și erorile.

Tema	Tema 8 din planul disciplinei (examene.pe.ub.ro), programarea pe partea de server, adaptată la Symfony
Durata	2 ore
Aveți nevoie de	proiectul `daw-template-ti-info-2026-2027` pornit, browserul și Postman
La final aveți	fișierul `HelloController.php` cu trei endpoint-uri funcționale, salvat pe GitHub pe branch-ul vostru

1. Obiective

La finalul laboratorului veți putea:

- explica ce face un controller și cum leagă Symfony o adresă de o funcție PHP;
- scrie endpoint-uri care întorc JSON, cu parametri în cale și în interogare (`?n=5`);
- testa un API în browser, în Swagger și în Postman;
- alege codul de stare HTTP potrivit (200, 400, 404) și găsi cauza unei erori în jurnal.

2. Pe scurt: cererea, ruta, controllerul, răspunsul

Fiecare cerere HTTP are o **metodă** (`GET`, `POST`, `DELETE` …) și o **adresă** (`/api/products`). Symfony caută **ruta** care se potrivește și apelează funcția PHP asociată, numită **acțiune**, dintr-o clasă numită **controller** [3]. Acțiunea primește datele cererii și întoarce un **răspuns**: în aplicația noastră, JSON, pe care frontend-ul React îl afișează [1], [2].

Ruta se declară chiar deasupra funcției, cu un **atribut** PHP 8: `#[Route('/api/hello/{name}', methods: ['GET'])]`. Partea `{name}` este un **parametru de cale**: ce scrie utilizatorul acolo ajunge în variabila `$name`. Răspunsul are și un **cod de stare**, care spune clientului rezultatul: `200` (reușită), `400` (cerere greșită), `404` (nu există) [4], [5].

3. Pornirea

1. Porniți **Docker Desktop**. Deschideți folderul `daw-template-ti-info-2026-2027` în editor și terminalul integrat.
2. Verificați pe ce branch sunteți: `git branch` marchează ramura curentă cu `*` (sau `git status`, pe primul rând: `On branch …`). Doar dacă nu sunteți deja pe branch-ul vostru, treceți pe el; apoi porniți containerele (înlocuiți `popescu.ion`):

```
git branch                # ramura curenta are * in fata
# doar daca NU sunteti deja pe branch-ul vostru:
git checkout popescu.ion   # sau: git switch popescu.ion
docker compose -f backend/compose.yaml up -d
docker compose -f frontend/compose.yaml up -d
```

3. Deschideți `backend/app/src/Controller/HealthController.php` și recunoașteți în el atributul `#[Route(...)]` și apelul `$this->json(...)`. Controllerul nostru va urma același tipar.

4. Primul controller

4.1 Crearea fișierului

Aveți două variante. Oricare ați alege, la final fișierul trebuie să conțină exact codul de la pasul 4.2.

- cu generatorul Symfony: `docker exec -it daw-php php bin/console make:controller HelloController` (creează fișierul cu un exemplu, pe care îl înlocuiți integral; dacă a creat și un șablon în `templates/`, acesta nu ne trebuie);
- manual: în `backend/app/src/Controller/` creați fișierul `HelloController.php`.

> **Atenție.** Numele fișierului trebuie să fie identic cu numele clasei (`HelloController`), inclusiv literele mari, iar fișierul trebuie să fie în folderul `src/Controller/`. Altfel, Symfony nu găsește clasa.

4.2 Codul complet

Scrieți în `HelloController.php` prima parte: declarațiile și primele două acțiuni.

```
<?php

namespace App\Controller;

use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\JsonResponse;
use Symfony\Component\HttpFoundation\Request;
use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\Routing\Attribute\Route;

final class HelloController extends AbstractController
{
  #[Route('/api/hello/{name}', name: 'api_hello', methods: ['GET'])]
  public function hello(string $name): JsonResponse
  {
    return $this->json(['message' => "Salut, $name!"]);
  }

  #[Route('/api/time', name: 'api_time', methods: ['GET'])]
  public function time(): JsonResponse
  {
    $now = new \DateTimeImmutable();

    return $this->json([
      'date' => $now->format('Y-m-d'),
      'time' => $now->format('H:i:s'),
      'timezone' => $now->getTimezone()->getName(),
    ]);
  }
```

Continuați, în aceeași clasă, cu a treia acțiune și acolada de închidere a clasei:

```
  #[Route('/api/numbers', name: 'api_numbers', methods: ['GET'])]
  public function numbers(Request $request): JsonResponse
  {
    // ?n=5 ; dacă lipsește, folosim 5
    $n = filter_var($request->query->get('n', 5), FILTER_VALIDATE_INT, [
      'options' => ['min_range' => 1, 'max_range' => 100],
    ]);

    if ($n === false) {
      return $this->json(
        ['error' => 'n trebuie să fie un număr întreg între 1 și 100.'],
        Response::HTTP_BAD_REQUEST
      );
    }

    $numbers = range(1, $n);

    return $this->json([
      'n' => $n,
      'numbers' => $numbers,
      'sum' => array_sum($numbers),
      'squares' => array_map(fn (int $x) => $x * $x, $numbers),
    ]);
  }
}
```

De ce. `$this->json(...)` transformă un tablou PHP în JSON și setează antetul `Content-Type: application/json`. Al doilea argument este codul de stare; implicit este `200`. `filter_var(..., FILTER_VALIDATE_INT, ...)` întoarce numărul dacă textul primit este un întreg între 1 și 100, altfel `false`. **Nu avem încredere în datele primite de la client**: le verificăm înainte să le folosim.

5. Testarea endpoint-urilor

5.1 În browser

Adresa	Ce ar trebui să vedeți	Cod
http://localhost:8080/api/hello/Ana	`{"message":"Salut, Ana!"}`	200
http://localhost:8080/api/time	data, ora și fusul orar al serverului	200
http://localhost:8080/api/numbers?n=5	numerele 1–5, suma 15 și pătratele	200
http://localhost:8080/api/numbers?n=abc	mesajul din cheia `error`	400
http://localhost:8080/api/hello	pagina de eroare: nu există rută fără nume	404

Sfat. Codul de stare se vede în **F12 → Network**, în coloana *Status*. Dacă ora diferă de cea locală, serverul folosește probabil fusul orar UTC; cheia `timezone` vă spune care este.

5.2 Lista rutelor

```
docker exec -it daw-php php bin/console debug:router
docker exec -it daw-php php bin/console debug:router api_numbers
```

Rezultat. Prima comandă afișează toate rutele, cu metoda și adresa; printre ele, `api_hello`, `api_time` și `api_numbers`. A doua afișează detaliile unei singure rute, după nume.

5.3 În Swagger și în Postman

1. Deschideți http://localhost:8080/api/doc. Căutați noile endpoint-uri, apăsați **Try it out**, completați `name` și apăsați **Execute**. Swagger arată adresa apelată, codul și corpul răspunsului.
2. În Postman: **New → HTTP Request**, metoda `GET`, adresa `http://localhost:8080/api/numbers`. În tab-ul **Params**, adăugați cheia `n` cu valoarea `10` și apăsați **Send**. Încercați apoi `n` = `0` și `n` = `abc`.
3. Salvați cererile (**Save**) în colecția importată în laboratorul 01, ca să le aveți pentru proiect.

> **Rezultat.** Pentru `n=10`, Postman arată *Status: 200 OK* și suma `55`. Pentru `0` sau `abc`, *Status: 400 Bad Request* și mesajul de eroare. Dacă noile rute nu apar în Swagger, verificați-le în browser și în Postman; documentarea lor detaliată o vom face într-un laborator viitor.

6. Când ceva nu merge: unde se văd erorile

Faceți intenționat o greșeală: în acțiunea `time`, scrieți `formatt` în loc de `format` și reîncărcați http://localhost:8080/api/time. Apare o pagină de eroare (cod `500`), cu mesajul, fișierul și linia problemei. Același mesaj, cu mai multe detalii, ajunge în jurnalul aplicației, fișierul `backend/app/var/log/dev.log`. Ultimele rânduri se pot vedea și din terminal:

```
docker exec -it daw-php tail -n 30 var/log/dev.log
```

Corectați greșeala și verificați că endpoint-ul funcționează din nou.

> **Sfat.** Citiți eroarea de sus în jos: prima linie spune **ce** s-a întâmplat, iar fișierul și numărul liniei spun **unde**. Majoritatea erorilor de început sunt un `;` lipsă, o acoladă neînchisă sau un `use` uitat.

7. Exerciții

1. Adăugați endpoint-ul `GET /api/sum?a=3&b=4`, care întoarce `{"a": 3, "b": 4, "sum": 7}`, cu răspuns `400` dacă `a` sau `b` lipsește ori nu este număr întreg.
2. Restricționați `{name}` la litere: în atributul rutei adăugați `requirements: ['name' => '[A-Za-z]+']`. Ce cod primiți acum la `/api/hello/123`? De ce?
3. Modificați `/api/numbers` astfel încât să întoarcă doar numerele pare de la 1 la `n` (indiciu: `array_filter`, apoi `array_values`).
4. ★ (opțional) Endpoint-ul `GET /api/factorial/{n}`, cu `n` între 0 și 20, care întoarce n! și `400` în afara intervalului.
5. ★ (opțional) În `frontend/app/src/api.js`, după modelul funcțiilor existente, adăugați `getHello(name)`, care cere `/api/hello/{name}`, și afișați mesajul pe pagina **Despre**.

8. Salvarea lucrului și oprirea

```
git status
git add .
git commit -m "Laboratorul 06: HelloController cu trei endpoint-uri JSON"
git push origin popescu.ion
# sau doar: git push (dupa ce ati facut o data git push -u)
docker compose -f backend/compose.yaml stop
docker compose -f frontend/compose.yaml stop
```

> **Push.** `git push origin popescu.ion` este forma completă: trimite branch-ul vostru la `origin`, adresa proiectului de pe GitHub. Forma scurtă, `git push`, merge doar dacă ați făcut anterior, o dată, `git push -u origin popescu.ion`; `-u` leagă branch-ul local de cel de pe GitHub.

9. Ce arătați la final

- cele trei endpoint-uri în browser, inclusiv răspunsul `400` la un `n` invalid;
- aceeași cerere în Postman, cu codul de stare vizibil, și ieșirea comenzii `debug:router`;

- exercițiile 1–3 rezolvate și commit-ul vizibil pe GitHub, pe branch-ul vostru.

Resurse pentru exersare

- **W3Schools – PHP:** https://www.w3schools.com/php/
- **W3Schools – PHP și JSON:** https://www.w3schools.com/php/php_json.asp
- **Tutorial PHP în română – introducere (video):** https://www.youtube.com/watch?v=JaTO46oiCEM
- **Ghid PHP în română:** https://php.punctsivirgula.ro/
- **Symfony (freeCodeCamp, video în engleză):** https://www.youtube.com/watch?v=Bo0guUbL5uo
- **Postman pentru începători (video în engleză):** https://www.youtube.com/watch?v=VywxIQ2ZXw4
- **Symfony – rutare (documentație):** https://symfony.com/doc/current/routing.html

Bibliografie

[1] E. Pruteanu și I. Furdu, *Modele, tehnologii, servicii și aplicații integrate de tip R.I.A. pentru web*. Bacău: Alma Mater, 2022.

[2] S. Buraga, *Tehnologii Web*. București: Matrix Rom, 2001.

[3] Symfony SAS, *Symfony Documentation – Controller; Routing*. symfony.com/doc/current/controller.html

[4] R. T. Fielding, *Architectural Styles and the Design of Network-based Software Architectures*, teză de doctorat, University of California, Irvine, 2000.

[5] Mozilla, *MDN Web Docs – HTTP response status codes*. developer.mozilla.org

[6] C.-M. Papară, *Metodologie de lucru și template de proiect – Programare Web / Dezvoltarea Aplicațiilor Web*, UVAB, 2026.